

Solving the Subcircuit Matching Problem by Using Encoded Matrix Method

Chia Shin Ou and R. C. T. Lee

Department of Communication Engineering

National Chi Nan University, Puli, Nantou Hsien, 545, Taiwan ROC

csou@alg.csie.ncnu.edu.tw

rctlee@ncnu.edu.tw

Abstract

The problem of finding subcircuit in a larger main circuit is an important issue recently. In this problem, we are given a main circuit and a subcircuit. Our job is to find all occurrences of the subcircuit in the main circuit. In this paper, we propose a method to solve the subcircuit matching problem by encoding a circuit into a connection matrix consisting of positive integers where each row and column corresponding to a component in the circuit. Our approach eliminates a row and its corresponding column in the matrix of the main circuit which cannot be any candidate of any component in the subcircuit. We have a mechanism to extract subcircuits from the surviving matrix. It is proved in the paper that our algorithm is sound and complete.

1 Introduction

The subcircuit finding problem is defined as follows: We are given a subcircuit S , and a main circuit G , which is usually much larger than the subcircuit. Our job is to determine whether S exists in G . Finding a subcircuit in the main circuit is a very important issue in analog circuit design because we often have to modify the subcircuit which may exist every where in the main circuit. Some research has been done in this field [1, 2, 3, 4]. Chang, Tzeng, and Lee use a recursive graph identification scheme and a weighted value to each node in graph to reduce the run time[1]. SubGemini[2] uses the partitioning and relabeling algorithm to find all subcircuits in large circuit. SUBGEN[3] describes an approach by using genetic algorithm, and Zhang and Wunsch II provide an algorithm for the subcircuit finding algorithm by using fuzzy attributed graph[4]. Our approach is to encode a circuit into a symmetrical matrix containing positive integers. We then provide four rules to

eliminate entries in the matrix corresponding to G . We also provide a mechanism to determine whether the surviving matrix contains integer number of submatrices corresponding to S . If this is the case, we have found matchings.

2.1 The Main Idea of Our Method

In our paper, the components in the circuits are limited to eight kinds, as illustrated in Fig. 1.

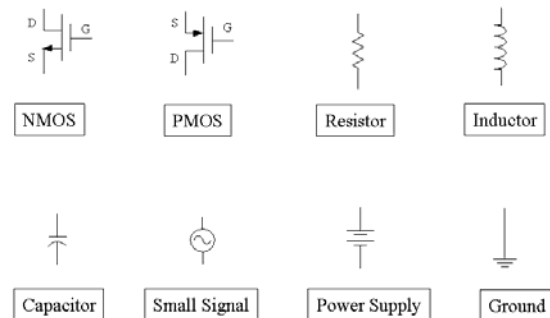


Fig.1

In an analog circuit, connections are made between two terminals. For example, the drain of a transistor may be connected to the source of another transistor. In the first step, if a terminal x of a component is connected to a terminal y of another component, $V(x, y) = 1$ and $V(x, y) = 0$ if otherwise. Some examples are in Fig. 2.

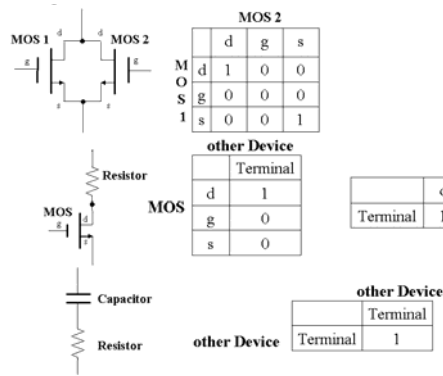


Fig. 2

Up to now, we showed how we encode the connection of terminals. Since two terminals are either connected or not connected, the subsequent matrices will contain 1's and 0's. Fig. 3 shows a circuit. Its corresponding terminal connection matrix is shown in Fig. 4.

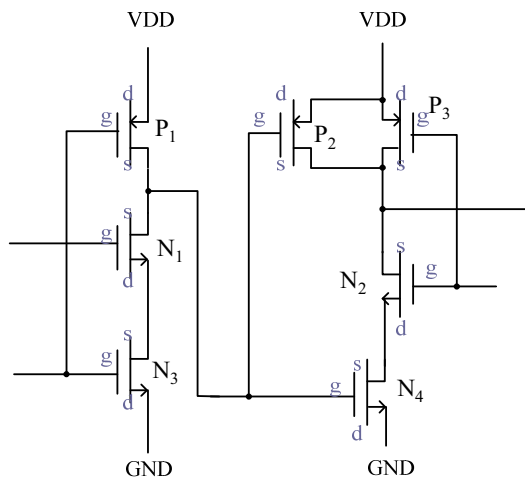


Fig. 3

	P ₁	P ₂	P ₃	N ₁	N ₂	N ₃	N ₄	VDD	GND
P ₁	d	g	s	d	g	s	d	g	s
P ₂	g	1	0	0	0	0	0	0	0
P ₃	g	0	0	0	0	0	0	0	0
N ₁	g	0	0	0	0	0	0	0	0
N ₂	s	0	0	0	0	0	0	0	0
N ₃	s	0	0	0	0	0	0	0	0
N ₄	s	0	0	0	0	0	0	0	0
VDD	0	0	0	0	0	0	0	1	0
GND	0	0	0	0	0	0	0	0	1

Fig. 4

But, it will become much more efficient if we encode the matrices which show the connection of components, instead of terminals of components. As can be seen, the encoded matrices will contain positive integers, not only 1's and 0's. In our algorithm, we encode the matrices as follows.

1. MOS 1 and MOS 2
2. MOS and other Component
3. Other Component 1 and other Component

Other components are resistor, inductor, capacitor, voltage source, and ground. We now illustrate our encoding method by using examples.

Case 1: Mos1 and Mos2

Suppose the source and drain of Mos1 are connected to the source and drain of Mos2 respectively. The connection matrix between Mos1 and Mos2 is shown in Fig. 5 and we encode the connection matrix into a positive integer which is shown in Fig. 6.

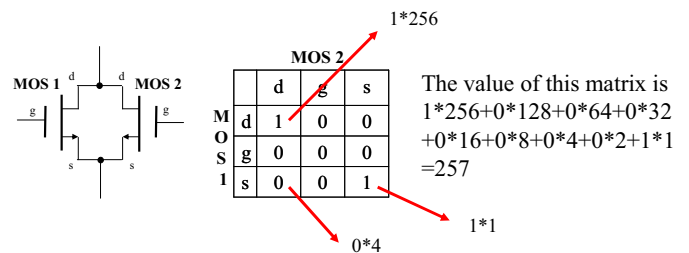


Fig. 5

MOS 2
MOS 1 257

Fig. 6

Case 2: Mos and Other Components

Suppose the drain of Mos is connected to a terminal of a resistor. There are two kinds of connection matrices in this case which are shown in Fig. 7 and we encode the connection matrices into positive integers which are shown in Fig. 8.

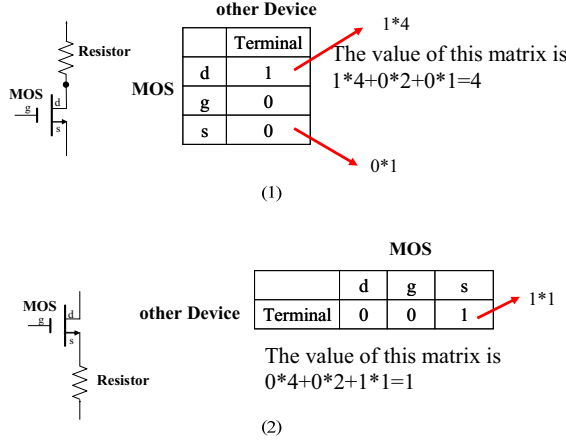
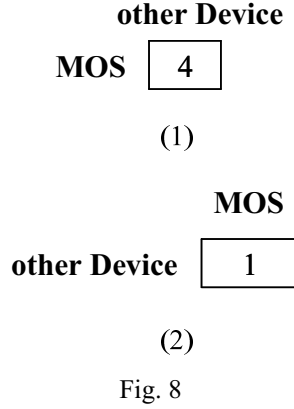


Fig. 7



Case 3: Two Other Components.

Suppose that a terminal of a resistor is connected to a terminal of a capacitor. The connection matrix between resistor and capacitor is shown in Fig. 9 and we encode the connection matrix into a positive integer which is shown in Fig. 10.

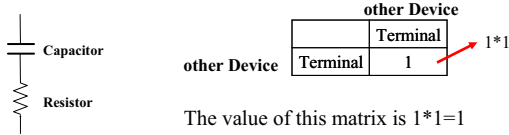


Fig. 9

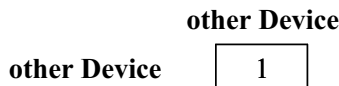


Fig. 10

Through this way, the two matrices encoding the terminals in Fig. 11 are now transformed into matrices encoding the

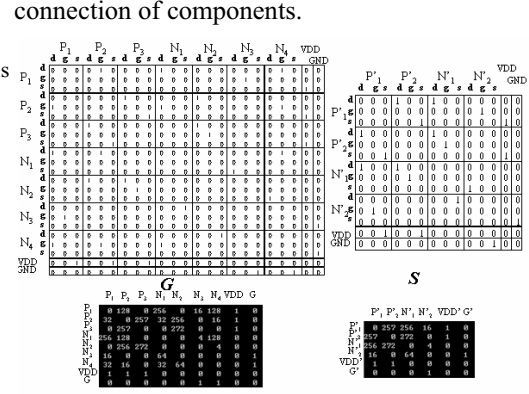


Fig. 11

2.2 The Checking Rules of Our Algorithm

We denote the encoded matrix for the main circuit $G(S)$ as $M(G)(M(S))$. Our algorithm is an algorithm which checks whether an entry in $M(G)$ can be a candidate for an entry in $M(S)$. If an entry x of $M(G)$ cannot be the candidate of any entry y in $M(S)$, x will be eliminated.

Rule 1: Suppose we consider a component x in S and a component y in G . Let the number of some type of components connected to x in S be a and the number of the same type of components connected to y be b . If y is a candidate for matching x , a must be smaller than or equal to b .

We show an example in Fig. 12.

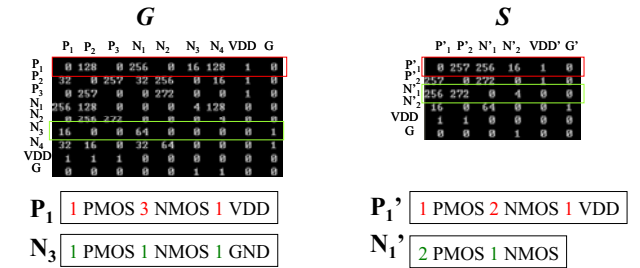


Fig. 12

In this case, P_1 is connected to one PMOS, three NMOS's and one VDD, and P_1' is connected to one PMOS, two NMOS's and one VDD. P_1 is a candidate for matching P_1' . In another case, N_3 is connected to one PMOS, one NMOS and one GND, and N_1' is connected to two PMOS, one NMOS. N_3 is not a candidate for matching N_1' .

Rule 2: Suppose a component x is connected to a component y in S and a component x' is connected to a component y' in G where x and x' are the same type and y and y' are the same type. If x' and y' are candidates for x and y respectively, $V(x', y')$ must be greater than or equal to $V(x, y)$.

It is not easy to comprehend that $V(x', y')$ can be larger than $V(x, y)$. In Fig. 13, the circuit in Fig. 13(a) has more connections between the two transistors. But they are obviously candidates for the two transistors in Fig. 13(b).

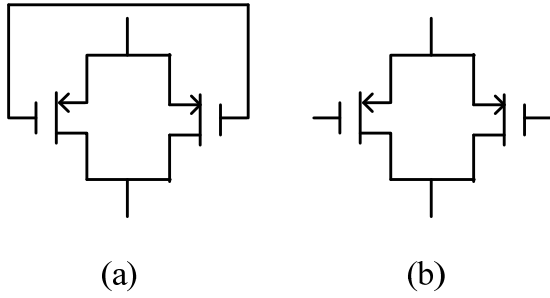


Fig. 13

There is a very important point concerning the application of Rule 2. Let x and y be components in S , x' and y' be components in G , x and x' are of the same type and y and y' are of the same type. For components x' and y' to be candidates of x and y respectively, each connection between a terminal in x and a terminal in y must exist between their corresponding terminals in x' and y' .

For example, if the drain of x is connected to the source of y , the drain of x' must also be connected to the source of y' . Note that Rule 3 only stipulates that a connection between x and y must exist in x' and y' , but not versa. Since we have already encoded the connections between terminals into $V(x, y)$ and $V(x', y')$, We can state the above rule as Rule 3 as follows:

Rule 3: Let x and y be components in S , x' and y' be components in G , x and x' are of the same type and y and y' are of the same type. For components x' and y' to be candidates of x and y respectively, if $V(x, y) \bmod 2^i$ is equal to 1 where $i = 1$,

$2, \dots, \lceil \log_2 V(x, y) \rceil$, $V(x', y') \bmod 2^i$ must be 1.

Rule 4: Let x be a component in S and x' be a component in G . If x matches x' , each component connected to x must match at least one component connected to x' and the matching is a one-to-one matching.

Our algorithm, as indicated before, checks every row of the matrix for the main circuit. If a particular row in $M(G)$ is found to be no candidate of any row in $M(S)$, we delete this row and its corresponding column in $M(G)$. The surviving matrix is denoted as $M'(G)$. Consider Fig. 14. $M(S)$ and $M'(G)$ are shown in Fig. 15. Note the following: Although P_1' matches P_1 and N_2' matches N_1 , $\{P_1', N_2'\}$ is not match of S . Similarly, $\{P_2', N_1'\}$ is neither a match of S . Only $\{P_1', N_1'\}$ and $\{P_1', N_1'\}$ are solutions. Thus we need a rule to find solutions in the surviving matrix $M'(G)$.

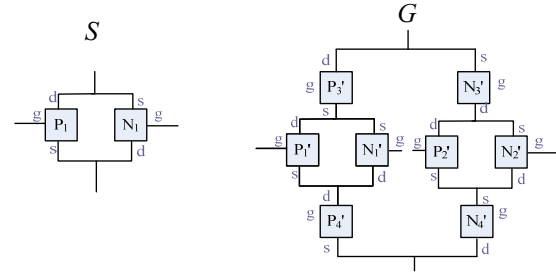


Fig. 14

	P_1	N_1
P_1	0	127
N_1	127	0

	P_1'	P_2'	N_1'	N_2'
P_1'	0	0	0	127
P_2'	0	0	0	0
N_1'	127	0	0	0
N_2'	0	127	0	0

Fig. 15

Rule 5: Let $S = \{x_1, x_2, \dots, x_n\}$ and $S' = \{x_1', x_2', \dots, x_n'\}$ in G . If S' matches S , $V(x_i', y_j') \geq V(x_i, x_j)$

Let us apply this rule to the matrices in Fig. 15. We can see, from Fig. 16(a) that $\{P_1', N_1'\}$ matches $\{P_1, N_1\}$ while, from Fig. 16(b) that $\{P_1', N_2'\}$ does not match $\{P_1, N_1\}$.

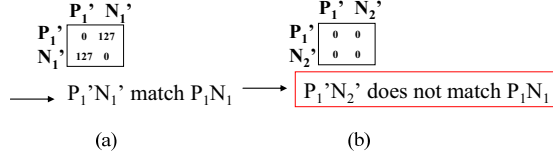


Fig. 16

2.3 Our Algorithm

Subcircuit matching algorithm

Input : A main circuit G and a subcircuit S

Output : All of the locations of subcircuits in G matching S .

Step 1. Transform S and G into their connection matrices.

Step 2. Encode the matrices. into $M(S)$ and $M(G)$.

Step 3. For each row x of $M(G)$, check whether x is a candidate of a row y in $M(S)$. Delete x and its corresponding columns if and only if it is found, by using Rules 1 to 4, that x is not a candidate of any row y in $M(S)$. If no row is deleted in this step, go to Step 5; otherwise, go to Step 4.

Step 4. If $M(G)$ is smaller than $M(S)$, end the algorithm and report that there is no subcircuit matching in G . Otherwise, go to Step 3.

Step 5. If solutions are found after application of Rule 5, report all such solutions in G ; otherwise, report that no solution is found.

3. The Soundness and Completeness of Our Algorithm

We first show the soundness of our algorithm. By soundness, we mean that every solution we find is a correct one. Suppose otherwise. Then there must be a case where a component $x'(y')$ in G is matched to a component $x(y)$ in S and the connection between x and y is different from that between x' and y' . But this is impossible

because of Rule 5 and Rule 3.

By completeness, we mean that every solution in G is found by our algorithm. Suppose a subcircuit A in G matching S exists and is not found by algorithm. There must exist a component x' of A which is deleted by our algorithm. But, our algorithm deletes x' if and only if x' is not a candidate for any component in S . Thus x' does not match any component of S . Thus A cannot match S , a contradiction.

4. Experiments

We wrote a C language problem to test and verify our algorithm. Since many circuits are in the form of a HSPICE program, we have also developed a C language program to transform a HSPICE program defining a circuit into the connection matrix corresponding to this circuit.

Experiment 1

The main circuit of the first experiment is shown in Fig. 17(a) and we found the subcircuit which is shown in Fig. 17(b). All of the circuits can be found in [2]. The time needed is 0.013 second.

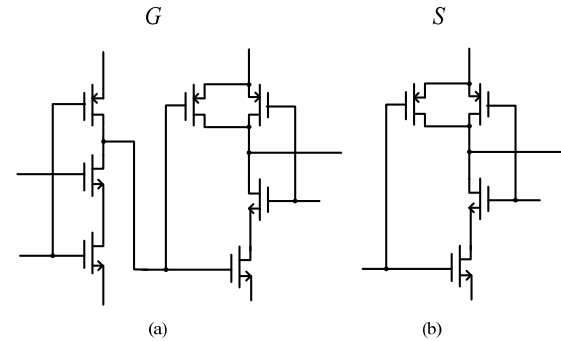


Fig. 17

Experiment 2

The main circuit of the second experiment is shown in Fig. 18 and we found the subcircuits shown in Fig. 19(a) and Fig. 19(b). There are 2 subcircuits of Fig. 19(a) in the main circuit and we found them all. Similarly, we found 2 subcircuits in Fig. 19(b) in the main circuit. The time needed for the subcircuit in Fig. 19(a) is 0.026 second and that needed for the subcircuit in Fig. 19(b) is 0.019 second.

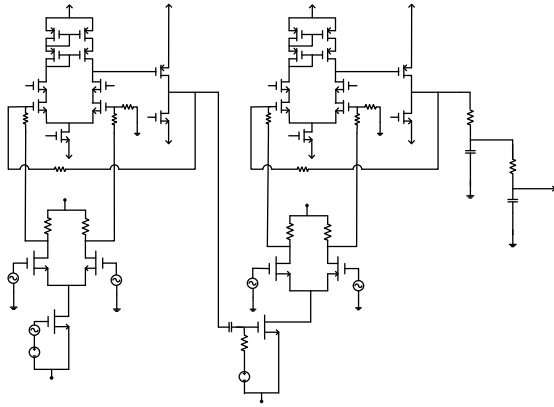


Fig. 18

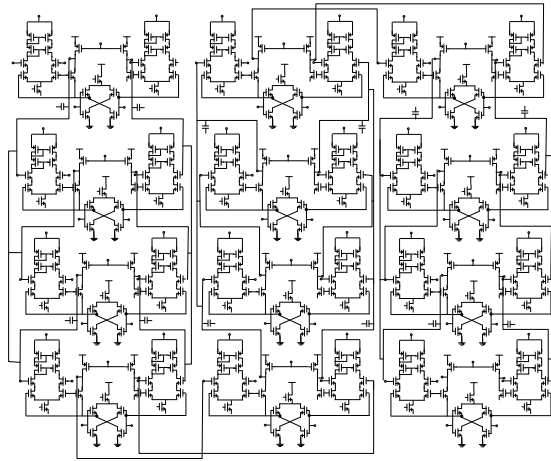


Fig. 20

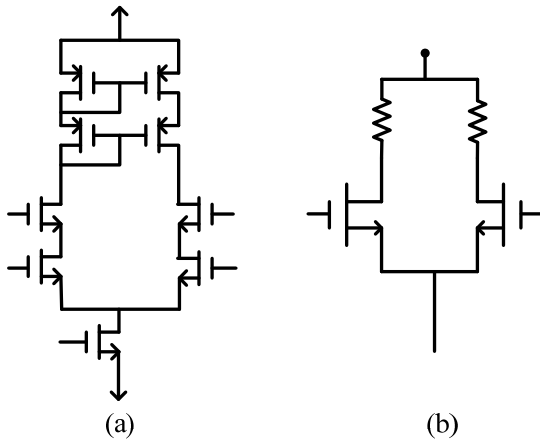


Fig. 19

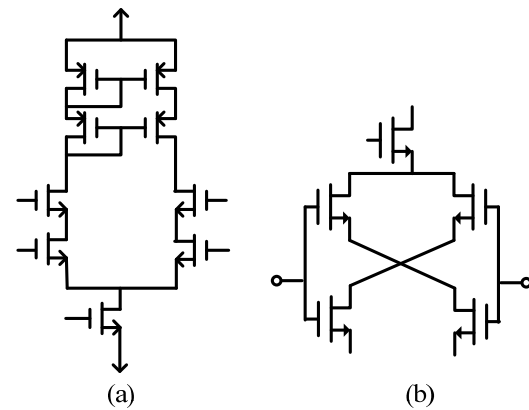


Fig. 21

Experiment 3

The main circuit of the third experiment is shown in Fig. 20 and we found the subcircuits shown in Fig. 21(a) and Fig. 21(b). There are 24 subcircuits of Fig. 21(a) in the main circuit and we found them all. Similarly, we found 12 subcircuits in Fig. 21(b) in the main circuit. The time needed for the subcircuit in Fig. 21(a) is 41.720 seconds and that for the subcircuit in Fig. 21(b) is 12.360 seconds.

Experiment 4

The main circuit of the fourth experiment is shown in Fig. 22 and we found all of the subcircuits shown in Fig. 23. There are 5 subcircuits of Fig. 23 in the main circuit and we found them all. The time needed is 6.240 seconds.

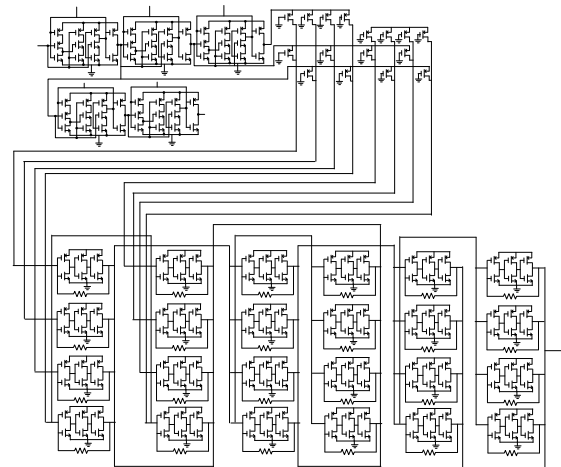


Fig. 22

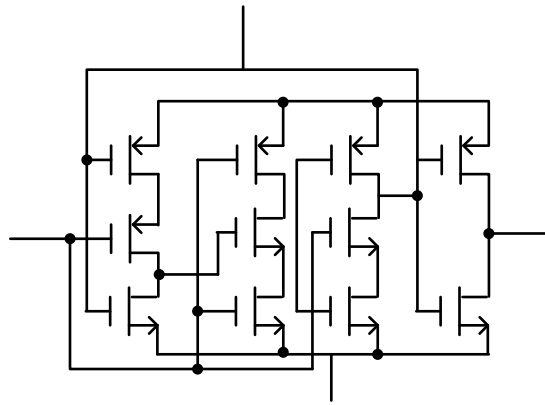


Fig. 23

provided us with the main circuit in Experiment 4.

5. Conclusion

In the paper, we proposed an algorithm to solve the subcircuit matching problem. Our algorithm transforms circuits into matrices with positive integers. By deleting non-candidates of components in the matrix corresponding to the main circuit, we can find all subcircuits in the main circuit. Besides, all subcircuits which we find are correct.

6. References

- [1] W. S. Chang, S. D. Tzeng, and C. Y. Lee, "Novel Subcircuit Extraction Algorithm by Recursive Identification Scheme", IEEE International Symposium on Circuits and Systems, pp. 491-494, 2001.
- [2] Miles Ohlrich, Carl Ebeling, Eka Ginting, and Lisa Sather, "SubGemini: Identifying SubCircuits using a Fast Subgraph Isomorphism Algorithm", ACM/IEEE Design Automation Conference, pp. 31-37, June 1993.
- [3] N. Vijaykrishnan and N. Ranganathan, "SUBGEN: A Genetic Approach for Subcircuit Extraction.", 9th. International Conference on VLSI Design, January 1996.
- [4] Nian Zhang and Donald C. Wunsch II, "A Fuzzy Attributed Graph Approach to Subcircuit Extraction Problem", The IEEE International Conference on Fuzzy Systems, St. Louis, MO, May 25-28, 2003.

Acknowledgements: We are grateful to Professor Jenn-Chyou Bor of the National Tsing Hua University who provided us with the main circuit in Experiment 3 and Mr. H. Y. Fan who